

AI Blog Nest API

Architecture Overview & Technical API Documentation

1. System Architecture

OVERVIEW

The AI Blog Nest API is built using NestJS, leveraging modular architecture, strict TypeScript typing, and seamless AI integration for automated content generation, dynamic optimization, and scalable content management.

Backend Framework

NestJS with dynamic module loading, dependency injection, and integrated OpenAPI/Swagger documentation.

AI Orchestration

Automated content drafting, title generation, and SEO optimization powered by Large Language Models.

Database Layer

PostgreSQL managed via Prisma ORM for type-safe database queries and migration management.

Authentication

JWT-based bearer authentication with role-based access control (RBAC) for API security.

2. Key Capabilities

FEATURES

- Automated Post Generation:** Create full-length blog posts using specific topics, tone, and targeted keywords.
- SEO & Metadata Processing:** Auto-generate search-engine optimized meta titles, descriptions, and tags.
- Content Lifecycle Management:** Manage posts through Draft, Scheduled, and Published states smoothly.
- Integrated Analytics:** Monitor views, AI generation token consumption, and engagement performance metrics.

3. Core Endpoints

API REFERENCE

Endpoint	Path	Description
POST	/api/v1/ai/generate	Triggers AI workflow to generate blog content based on prompt settings.
GET	/api/v1/posts	Retrieves paginated list of blog posts with search and status filters.
GET	/api/v1/posts/:id	Fetches a single blog post by its unique identifier.
POST	/api/v1/posts	Manually creates or overrides a blog post entry in the system.
DELETE	/api/v1/posts/:id	Removes a post entry and purges associated media indices.

4. Example Request Payload

JSON

```
{ "prompt": "The Future of Serverless Architectures in 2026", "targetAudience": "Back end Engineers", "tone": "Technical & Analytical", "wordCount": 1200, "seoKeywords": ["serve rless", "NestJS", "cloud computing"], "publishDirectly": false }
```

5. Integration & Deployment

DEVOPS

The API is fully containerized using Docker and ready for deployment on Kubernetes or serverless container platforms. Environment configurations handle API key rotations and database connection pooling seamlessly.

